

```

lowererrors = [abs(outcome - meanlower) for outcome in loweroutcomes]
highererrors = [abs(outcome - meanhigher) for outcome in
higheroutcomes]

total_error = sum(lowererrors) + sum(highererrors)

if total_error < lowest_error:
    best_split = split_candidate
    lowest_error = total_error
    best_lowermean = meanlower
    best_highermean = meanhigher
return(best_split, lowest_error, best_lowermean, best_highermean)

```

[Listing 9-2](#): A function that finds the best point at which to split a variable for a branch point of a decision tree

In this function, we use a variable called `pctl` (short for *percentile*) to loop through every number from 0 to 100. In the first line of the loop, we define a new `split_candidate` variable, which is the `pctl`-th percentile of the data. After that, we go through the same process we used in [Listing 9-2](#). We create a list of the happiness levels of people whose `sclmeet` values are less than or equal to the split candidate, and the happiness levels of people whose `sclmeet` values are greater than the split candidate, and we check the errors that come from using that split candidate. If the error sum from using that split candidate is smaller than any of the error sums from using any previous split candidate, then we redefine the `best_split` variable to be equal to `split_candidate`. After the loop completes, the `best_split` variable is equal to the split point that led to the highest accuracy.

We can run this function for any variable, as in the following example where we run it for `hhmb`, the variable recording the respondent's number of household members.

```

allvalues = list(ess.loc[:, 'hhmb'])
predictedvalues = list(ess.loc[:, 'happy'])
print(get_splitpoint(allvalues, predictedvalues))

```

The output here shows us the correct split point as well as the predicted happiness level for the groups defined by that split point: